Open-source security methodology

Network Security Assessment Roadmap

A step-by-step guide to checking a network from the outside in — external & internal.

Foundation Practitioner Advanced

Created by

Emmanuel Bairaktaris

linkedin.com/in/emmanuel-bairaktaris

IT Department · Field Manual

On-prem · Windows AD / Linux / Network

Open-source toolchain

00 How to use this manual F · Foundation

This is both a **teaching guide** and a **field manual**. Every technique is written in three layers so it serves the whole team:

> Concept What the step is and why it matters. Read this if you're learning.

> Do this The exact commands and scripts to run. Read this if you just need to execute.

> Go deeper Advanced options, edge cases and follow-ups for experienced staff.

Each section is tagged by difficulty — F · Foundation everyone should manage, P · Practitioner needs some background, A · Advanced is for senior staff or supervised learning. Use the legend above to focus a single tier.

Beginners, start in the lab

Don't run anything from Parts I–II against production until you've done it once in the practice lab (Section 2.4). Reading output correctly is a skill — break things in the lab, not in prod.

Customization placeholders

This manual uses variables in <ANGLE_BRACKETS>. Fill these in once for our environment and keep the master copy updated.

Placeholder	Meaning	Our value
<EXTERNAL_RANGES>	Public IP ranges / CIDRs we own	TODO
<DOMAINS>	Public domain names we own	TODO
<INTERNAL_SUBNETS>	Internal CIDRs, per VLAN	TODO
<AD_DOMAIN>	Active Directory domain	TODO
<DC_IP>	Domain Controller IP(s)	TODO
<SCANNER_HOST>	The machine you run scans from	TODO
<AUTHORIZED_BY>	Who signs off on active testing	TODO

Assumptions ⚙ revisit

- Testing is limited to infrastructure **we own or are explicitly authorized to test**.
- Scale, existing security stack (firewall / EDR / SIEM / IDS) and compliance drivers (PCI-DSS, ISO 27001, etc.) are **not yet specified**. Sections that depend on these carry a ⚙ customize marker.

01 Ground rules: authorization & safety F · Foundation

> Concept An assessment uses the same tools an attacker would. The only thing separating a security audit from an intrusion is **written authorization and a defined scope**. This is not bureaucracy — it protects you personally and keeps testing from causing an outage.

Do this

Before any active scanning:

1. **Get written authorization.** A short Rules of Engagement document signed by `<AUTHORIZED_BY>` stating what will be tested, the ranges/domains in scope, the time window, who is running it, and an emergency contact.
2. **Define scope explicitly.** List in-scope `<EXTERNAL_RANGES>` / `<INTERNAL_SUBNETS>`, and list **out-of-scope** systems (safety-critical, third-party hosted, shared tenancy).
3. **Confirm ownership of external targets.** Only scan public IPs/domains registered to us — scanning IPs you don't own can be illegal even if you think they're yours (shared hosting is a common trap).
4. **Notify the NOC / on-call** before internal scans, and record a baseline so you can attribute any disruption.
5. **Start non-disruptive.** Aggressive scans can crash fragile devices (old printers, ICS/OT, legacy appliances). Begin gently and escalate deliberately.

Go deeper Keep an evidence log: timestamp every command and save raw output. If a system misbehaves you need to prove exactly what you sent and when. Store logs securely — scan output is a map of your weaknesses. A good external reference for structure is the OWASP Web Security Testing Guide.

02 Toolkit setup F · Foundation

Concept All tools below are open-source. You can install them piecemeal, but the fastest path is a purpose-built security distro that ships them pre-packaged.

Recommended base

Kali Linux or Parrot OS Security — Debian-based, ship with the toolset. Run as a VM or a dedicated laptop. Alternatively a clean Debian/Ubuntu host where you install tools yourself (more control, better for a permanent scanning server).

Core install (Debian / Ubuntu base)

```
# Package-manager tools
sudo apt update && sudo apt install -y \
  nmap masscan dnsutils whois \
  nikto wafw00f whatweb \
  smbclient ldap-utils snmp \
  arp-scan netdiscover \
  tcpdump wireshark \
  hydra john hashcat \
  python3-pip pipx git

# Better installed from source / pipx
pipx install impacket        # AD/SMB scripting (GetUserSPNs.py, secretsdump.py)
pipx install netexec         # successor to CrackMapExec (command: nxc)
pipx install ssh-audit       # SSH server config auditing
pipx install dnsrecon        # DNS enumeration
```

Additional open-source tools

Tool	Purpose	Notes
Greenbone / OpenVAS (GVM)	Full vulnerability scanner	The open-source Nessus alternative. Best run as a dedicated VM.
Nuclei	Template-based vuln / misconfig scanner	Fast, huge community template set.
testssl.sh	TLS/SSL configuration testing	Single bash script.
theHarvester	OSINT: emails, subdomains, hosts	Passive external recon.
Amass	Attack-surface / subdomain mapping	OWASP project.
enum4linux-ng	SMB / Windows enumeration	Python rewrite of enum4linux.
kerbrute	Kerberos user enumeration	AD assessment.
BloodHound CE + bloodhound.py	AD attack-path mapping	Python or SharpHound collector.
Lynis	Linux/Unix host hardening audit	`sudo apt install lynis`.
OpenSCAP + SCAP content	CIS / STIG compliance scanning	`oscap` CLI.

Build a practice lab

> `Concept` You will misread output and misuse flags while learning. Do that where it's safe.

> `Do this` Stand up isolated VMs on a host-only network: a vulnerable target (Metasploitable or a VulnHub image), and a mini-AD — a Windows Server DC + client, or the GOAD ("Game of Active Directory") lab. Point your scanner at these until the commands feel routine.

Part I

External assessment

The attacker's view from the internet. Goal: know exactly what we expose and whether it's defensible.

03 Map the external footprint F · Foundation

Concept You can't protect assets you don't know you have. Attackers start by inventorying everything you expose — so should you.

Do this

Collect the authoritative list of `<EXTERNAL_RANGES>` and `<DOMAINS>` from whoever manages DNS/ hosting, then confirm ownership:

```
whois <EXTERNAL_RANGES>   # confirm the block is registered to us
whois <DOMAINS>           # registrar, expiry, contacts
```

Go deeper Cross-check against billing and asset records. "Shadow IT" — a forgotten VPS or marketing microsite — is a classic entry point precisely because nobody remembers to patch it.

04 Passive reconnaissance (OSINT) P · Practitioner

Concept *Passive* means you gather what's already public without touching our servers — no packets to us, nothing logged on our side. This mirrors an attacker's quiet first phase and often reveals subdomains, leaked emails and exposed services you didn't know were public.

Do this

```
# Emails, subdomains, hosts referencing our domain
theHarvester -d <DOMAINS> -b all

# Attack-surface / subdomain enumeration (passive mode)
amass enum -passive -d <DOMAINS>

# Certificate transparency logs reveal hidden subdomains — search crt.sh
```

Certificate-transparency lookups: crt.sh.

Go deeper Search for leaked credentials tied to our domains in public breach indexes, and check whether staff emails appear in known breaches (e.g. Have I Been Pwned). This informs password-reset and MFA priorities.

Reading it

Every subdomain returned is a potential target. Flag anything unexpected — `dev.`, `test.`, `old.`, `vpn.`, `admin.` — as non-production hosts are frequently unpatched.

05 DNS enumeration P · Practitioner

Concept DNS is the phone book of your external presence. Misconfigurations — open zone transfers, stale records — leak internal structure.

Do this

```
dnsrecon -d <DOMAINS>                    # records, subdomains, misconfigs
dig axfr <DOMAINS> @<their-nameserver>   # zone transfer test — should FAIL
```

Reading it

A successful zone transfer (`axfr` returns the whole zone) is a finding — it hands attackers your entire DNS layout. It should be refused.

06 Email security posture F · Foundation ⚙ customize

Concept SPF, DKIM and DMARC records decide whether attackers can spoof mail from your domain. Cheap to check, high impact for phishing resistance.

Do this

```
dig +short TXT <DOMAINS>           # look for the v=spf1 record
dig +short TXT _dmarc.<DOMAINS>    # look for the v=DMARC1 record
```

Background: dmarc.org.

Reading it

Missing SPF/DMARC, or DMARC set to `p=none`, means anyone can spoof your domain in email. Recommend moving toward `p=quarantine` / `p=reject` after a monitoring period.

07 External host discovery & port scanning P · Practitioner

Concept Now you actively probe in-scope external IPs to see which are alive and what ports answer. An open port is an exposed service, which is an attack surface. **This is active** — authorization and scope (Section 1) must be in place.

Do this

```
# Fast, wide sweep to find live hosts / open ports
sudo masscan <EXTERNAL_RANGES> -p1-65535 --rate 1000 -oL masscan-ext.txt

# Detailed follow-up with nmap (service + version detection)
sudo nmap -sV -sC -p<ports-from-masscan> -oA nmap-ext <target-ips>
```

Flags: `-sV` version detection, `-sC` default safe scripts, `-oA` save all formats. Start at `--rate 1000` (gentle) — never blast max rate at a production edge. Reference: nmap manual, masscan.

Go deeper Compare results to firewall rules. Every externally-open port should map to an intentional, documented rule. Undocumented exposure = misconfiguration or shadow IT.

Priority finding

Externally you should generally see *only* deliberately published services (443 on a web server, 25 on mail). Anything else — 3389 (RDP), 445 (SMB), 22 (SSH), database ports — reachable from the internet is a top-priority question to answer.

08 Service & web surface testing P · Practitioner

Concept Once you know what's listening, test how well it's configured — outdated software, weak TLS, exposed admin panels, known CVEs.

Do this

```
testssl.sh https://<host>    # TLS/SSL config and certificate health
nikto -h https://<host>      # web server misconfig / known-issue scan
whatweb https://<host>       # fingerprint the tech stack
wafw00f  https://<host>      # detect a web application firewall
nuclei -u https://<host>     # template checks for CVEs and misconfigs
```

Tools: Nikto, WhatWeb, wafw00f.

Reading it

`testssl.sh` flags weak protocols (SSLv3, TLS 1.0/1.1), weak ciphers and expired/self-signed certs. `nuclei` / `nikto` surface known-vulnerable versions and exposed paths — verify each before treating it as confirmed (see Section 19).

09 External vulnerability scan P · Practitioner ⚙ customize

Concept A vulnerability scanner (Greenbone / OpenVAS) automates checking every exposed service against a database of known vulnerabilities and produces a prioritized report — the backbone of a repeatable program.

Do this

1. Deploy Greenbone/OpenVAS as a VM and let the feeds sync (first sync is slow).
2. Create a target for `<EXTERNAL_RANGES>` and run a **Full and Fast** scan.
3. Export the report (PDF/CSV) for the analysis phase (Part III).

⚙ Customize

If we adopt a commercial scanner later, this section changes. For now, OpenVAS is the open-source standard.

Part II

Internal assessment

The view of an attacker who already has a foothold inside — a phished user, a rogue device, or malware. Internal networks are usually flatter and softer than the perimeter, so this is where the big findings live.

10 Position your scanner F · Foundation

> Concept Segmentation matters: what you can see depends on which VLAN you're plugged into. Run scans from a host on each relevant segment to test both reachability and segmentation.

> Do this Place `<SCANNER_HOST>` on a segment, note its IP/subnet, and record which other `<INTERNAL_SUBNETS>` it can reach. The reachability map itself is a segmentation finding.

11 Layer-2 / local discovery F · Foundation

> Concept On your local subnet you can discover hosts at the data-link layer, which finds devices even if they don't answer higher-level pings.

> Do this

```
sudo arp-scan --localnet              # everything on the local segment
sudo netdiscover -r <INTERNAL_SUBNETS>
```

> Go deeper Passive listening: Responder in **analyze-only** mode (`responder -A -I <iface>`) *listens* for LLMNR/NBT-NS/mDNS traffic without answering. Seeing this traffic means the network is vulnerable to name-poisoning credential theft — a very common weakness.

Do not poison production

Never run Responder in active/poisoning mode on production without explicit authorization. Analyze mode (`-A`) is the safe diagnostic.

12 Internal host & port sweep P · Practitioner

> Concept Same idea as external scanning, but internally you'll find far more — and the expectation flips: many internal services are normal, so you're hunting the *unexpected and the unpatched*.

> Do this

```
sudo nmap -sn <INTERNAL_SUBNETS> -oA internal-hostsweep   # live hosts
sudo nmap -sV -sC -O <INTERNAL_SUBNETS> -oA internal-services
```

`-O` adds OS detection — useful for spotting end-of-life systems (Windows 7 / Server 2008, unsupported Linux).

Reading it

Prioritize legacy/EOL operating systems, management ports open network-wide (RDP/SSH/WinRM), and databases or file shares reachable from user VLANs.

13 Windows & SMB enumeration P · Practitioner

`Concept` SMB (port 445) enumeration reveals shares, users and policy — and misconfigured shares are a top internal risk for data exposure and lateral movement.

`Do this`

```
enum4linux-ng -A <host>           # shares, users, OS, policies
smbclient -L //<host>/ -N         # list shares (-N = null/anonymous)
nxc smb <INTERNAL_SUBNETS>        # sweep SMB; flag signing & guest access
```

Tool: NetExec (`nxc`).

Priority findings

Anonymous/null access to shares = data exposure. SMB signing not required = network vulnerable to relay attacks. SMBv1 enabled anywhere = high priority; it should be disabled everywhere.

14 Active Directory review A · Advanced

`Concept` AD is the crown jewels — compromise of the domain usually means game over. This step maps how a foothold could escalate to domain-wide control. Everything here is **authorized defensive auditing** of `<AD_DOMAIN>`, to find and fix misconfigurations before an attacker does.

`Do this`

With valid, authorized domain credentials:

```
# User enumeration / weak Kerberos pre-auth (needs a userlist)
kerbrute userenum -d <AD_DOMAIN> --dc <DC_IP> users.txt

# Collect AD relationships, then review paths in BloodHound CE
bloodhound-python -d <AD_DOMAIN> -u <user> -p <pass> -ns <DC_IP> -c All

# Identify service accounts exposed to Kerberoasting (auditable misconfig)
GetUserSPNs.py <AD_DOMAIN>/<user>:<pass> -dc-ip <DC_IP>

# Identify accounts with Kerberos pre-auth disabled (AS-REP roastable)
GetNPUsers.py <AD_DOMAIN>/ -usersfile users.txt -dc-ip <DC_IP>
```

Tools: BloodHound CE, Impacket, kerbrute.

`Go deeper` BloodHound shows the shortest paths to Domain Admin — those paths *are* your remediation list. Service accounts with weak passwords and SPNs, and accounts with pre-auth disabled, are concrete fixes: rotate to strong passwords / gMSAs and re-enable pre-auth. The point is to close these, not exploit them.

Scope note

Cracking any hashes obtained (Section 18) is only for verifying policy strength on **our own** accounts, under authorization. Handle credential material as sensitive and destroy it after the assessment.

15 Internal vulnerability scan P · Practitioner ⚙ customize

Concept Run Greenbone/OpenVAS across `<INTERNAL_SUBNETS>`. Internal scans typically return far more than external ones because internal hosts are patched less aggressively.

Do this Create internal targets and run **authenticated** scans where possible (give the scanner read credentials) — authenticated scans see missing patches that unauthenticated scans miss. Export reports for Part III.

16 Host hardening audits P · Practitioner

Concept Vuln scans find *missing patches*; hardening audits find *insecure configuration* — weak SSH, permissive permissions, disabled logging. Both matter.

Do this

Linux / Unix:

```
sudo lynis audit system   # hardening index + specific warnings
ssh-audit <host>          # SSH ciphers / algorithms / config
```

Windows & general, via OpenSCAP + CIS content:

```
oscap xccdf eval --profile <cis-profile> --results out.xml <scap-content.xml>
```

Tools: Lynis, ssh-audit. Benchmarks: CIS Benchmarks.

⚙ Customize

Set the OpenSCAP profile to our chosen standard. Good compliance evidence if a framework is ever in scope.

17 Network device audits P · Practitioner

Concept Routers, switches and firewalls are high-value: one weak device can undo all host hardening. They're also frequently left on defaults.

Do this

```
snmpwalk -v2c -c public <device-ip>   # if 'public' works, that's a finding
ssh-audit <device-ip>                 # SSH config on the mgmt interface
sudo nmap -sV <device-ip>             # confirm only expected mgmt ports
```

Go deeper Highest value is a manual config review: export each device config and check for default/weak credentials, telnet or HTTP management, SNMP `public`/`private`, missing ACLs on management interfaces, and unencrypted management. Compare against the CIS Benchmark for the platform.

18 Traffic & segmentation validation A · Advanced

Concept Capturing traffic reveals cleartext protocols and validates that VLAN segmentation actually works.

Do this

```
# Capture (authorized, on a SPAN port or your own host)
sudo tcpdump -i <iface> -w capture.pcap
# Open capture.pcap in Wireshark; filter for telnet, ftp, http basic-auth
```

Tools: tcpdump, Wireshark.

Segmentation test: from a user VLAN, attempt to reach hosts you *shouldn't* (server / OT / management subnets) using the Section 12 sweeps.

Go deeper Password-policy verification: if auditing password strength on our own accounts under authorization, crack a controlled hash sample offline with hashcat / John the Ripper to measure real-world strength against policy. This measures policy effectiveness — it is not for gaining access. Treat all hash material as sensitive and delete it when done.

Finding

Successful reachability across segments that policy says should be blocked is a finding.

Part III

Analysis, reporting & remediation

Scanning produces data. Value comes from turning data into prioritized, tracked fixes — on a repeating cadence.

19 Triage & false positives P · Practitioner

Concept Automated scanners over-report. Every "critical" must be validated before it enters a report, or the team loses trust in the process.

Do this For each significant finding confirm: Is the service actually running and reachable? Is the vulnerable version actually deployed? Is there a compensating control (firewall, EDR — ⚙ customize)? Only validated findings advance.

20 Risk scoring & prioritization F · Foundation

Concept You can't fix everything at once. Prioritize by real risk, not just scanner severity.

Do this For each validated finding record its CVSS score, exposure (internet-facing vs internal-only), exploitability (public exploit available?) and asset value. A medium-severity flaw on an internet-facing server usually outranks a "critical" on an isolated test box.

21 Reporting F · Foundation ⚙ customize

Concept The report is the deliverable. It must serve both leadership (summary/risk) and engineers (how to fix).

Do this

Structure each report as:

1. **Executive summary** — overall posture and top risks, in plain language.
2. **Scope & methodology** — what was tested, when, how (reference this manual).
3. **Findings** — each with description, evidence, affected assets, risk rating and **specific remediation steps**.
4. **Appendix** — raw scan outputs, tool versions, timestamps.

⚙ Customize

If a compliance framework applies, add a control-mapping section (e.g. map findings to ISO 27001 / PCI-DSS controls).

22 Remediation & retest F · Foundation

Concept A finding isn't closed until it's fixed *and verified fixed*.

Do this Log every finding in the ticketing system with an owner and a due date (severity-based SLAs). After remediation, re-run the specific test to confirm closure. Keep before/after evidence.

23 Cadence & metrics F · Foundation

> Concept Security posture decays — new systems, new CVEs, config drift. A one-time assessment is nearly worthless; a *program* is what protects you.

> Do this
>
> Suggested cadence ⚙ customize:
>
> - External vuln scan — **monthly** (or continuous for the perimeter).
> - Internal vuln scan — **quarterly**.
> - Full internal assessment (Parts I–II) — **annually**, plus after any major change.
> - Config / hardening audits — **quarterly** or gated in change management.
>
> Track over time: open findings by severity, mean time to remediate, % of assets scanned, recurrence rate. These trends prove the program is working.

A Command quick reference

```
# DISCOVERY
whois <target>                        # ownership
theHarvester -d <domain> -b all       # passive OSINT
dnsrecon -d <domain>                  # DNS enum
arp-scan --localnet                  # L2 host discovery
nmap -sn <cidr>                       # ping/host sweep

# PORT / SERVICE SCANNING
masscan <cidr> -p1-65535 --rate 1000  # fast port sweep
nmap -sV -sC -oA out <targets>        # service+version+scripts

# WEB / TLS
testssl.sh https://<host>             # TLS audit
nikto -h https://<host>               # web misconfig
nuclei -u https://<host>              # CVE/misconfig templates

# WINDOWS / AD / SMB
enum4linux-ng -A <host>               # SMB/Windows enum
nxc smb <cidr>                        # SMB sweep (signing, guest)
bloodhound-python -d <dom> -u <u> -p <p> -ns <dc> -c All
GetUserSPNs.py <dom>/<u>:<p> -dc-ip <dc>   # Kerberoastable accounts

# HARDENING / DEVICES
lynis audit system                   # Linux hardening
ssh-audit <host>                      # SSH config
snmpwalk -v2c -c public <device>      # default SNMP check
oscap xccdf eval --profile <p> <content>   # CIS/STIG compliance
```

B Tool & reference links

Category	Resource
Distros	Kali Linux, Parrot OS
Scanning	Nmap, masscan
Recon / OSINT	theHarvester, Amass, dnsrecon, crt.sh
Web / TLS	testssl.sh, Nikto, Nuclei, WhatWeb, wafw00f
Vuln scanning	Greenbone / OpenVAS
Windows / AD	enum4linux-ng, NetExec, Impacket, kerbrute, BloodHound CE, bloodhound.py
Hardening	Lynis, OpenSCAP, ssh-audit
Traffic	Wireshark, tcpdump, Responder
Password audit	hashcat, John the Ripper
Frameworks	OWASP WSTG, CIS Benchmarks, CVSS
Practice labs	VulnHub, GOAD

C Glossary for newer staff

- **Port / service** — a numbered channel a program listens on (443 = HTTPS). Open ports are entry points.
- **Passive vs active recon** — passive uses only public info (no packets to the target); active sends probes to the target.
- **CVE / CVSS** — a CVE is a catalogued known vulnerability; CVSS is its 0–10 severity score.
- **Enumeration** — extracting detail (users, shares, versions) from a service.
- **Lateral movement / privilege escalation** — moving between machines / gaining higher rights once inside.
- **Kerberoasting** — abusing weak AD service-account passwords; auditable and fixable.
- **Segmentation** — dividing the network so a breach in one zone can't reach others.

D Safe-scanning checklist

- [ ] Written authorization signed by `<AUTHORIZED_BY>`
- [ ] Scope (in / out) confirmed against `<EXTERNAL_RANGES>` / `<INTERNAL_SUBNETS>`
- [ ] External targets confirmed as owned by us
- [ ] NOC / on-call notified; maintenance window if needed
- [ ] Fragile / OT / legacy systems identified and excluded or handled gently
- [ ] Start with gentle rates; escalate deliberately
- [ ] All commands and raw output logged with timestamps
- [ ] Scan data and credential material stored securely, deleted when done

---

This is a living document. Keep the placeholder table and the ⚙-marked sections up to date as our environment, tooling and compliance requirements evolve.